

Test-Time Training pour modèles auto-supervisés

TTT++ et ActMAD

Rayane KHATIM

TER M1 – Université Paris-Cité

Mars 2026



- 1 Contexte et objectif du TER
- 2 Article 1 : TTT++
- 3 Article 2 : ActMAD
- 4 Lien avec le TER
- 5 Conclusion

Contexte : auto-supervision et shift

- Pré-entraînement auto-supervisé sur un grand jeu non annoté.
- Fine-tuning supervisé sur un jeu annoté plus petit.
- Au test, distribution différente (bruit, météo, domaine) $\Rightarrow$ baisse de performance.

Test-Time Training (TTT)

- Adapter le modèle directement sur les données de test, sans labels.
- Utiliser une tâche auto-supervisée ou l'alignement de distributions internes.
- Objectif du TER : appliquer TTT à un modèle auto-supervisé pour la classification d'images.

TTT classique : principe et limites

- Modèle avec un encodeur partagé + tête tâche principale + tête SSL.
- Au test : on met à jour l'encodeur en optimisant uniquement la loss SSL.
- Problème : sous gros shift, cette mise à jour peut dégrader la performance de la tâche principale.

- **Feature alignment** : stocker moyenne et covariance des features d'entraînement.
- **Moment matching** au test : contraindre les features test à rester proches des features train.
- **SSL contrastive forte** (type SimCLR) pour que la tâche SSL soit bien corrélée à la tâche principale.
- **Queue dynamique** pour estimer les moments de manière stable avec de petits batches.

TTT++ : pipeline

- 1 Entraînement multi-tâche : $\text{loss} = \text{classification} + \lambda \text{loss contrastive}$.
- 2 Après entraînement : calcul de μ et Σ des features (encodeur et tête SSL) sur le train.
- 3 Test-time : $\text{loss totale} = \text{loss contrastive} + \text{pertes d'alignement de moments}$, mise à jour de l'encodeur.

TTT++ : résultats et messages clés

- Amélioration significative sur CIFAR-10C/100C, VisDA-C, CIFAR10.1.
- Montre que TTT *peut* échouer si la tâche SSL est mal choisie ou si on ne contrôle pas la distribution des features.
- Importance de la corrélation entre tâche principale et tâche SSL et du rôle des statistiques de features.

- Beaucoup de méthodes TTT supposent :
 - un entraînement multi-tâche spécial,
 - une architecture précise (BatchNorm),
 - ou une tâche de classification uniquement.
- Objectif ActMAD : méthode de TTT **générique**, indépendante de l'architecture et de la tâche.

- Considérer chaque activation $a_l(x)$ (canal et position spatiale) comme une variable aléatoire.
- Pour plusieurs couches l :
 - Offline : calcul de $\hat{\mu}_l$ et $\hat{\sigma}_l^2$ sur les données clean.
 - Online test : calcul de $\mu_l(B)$, $\sigma_l^2(B)$ sur le batch de test.
- Loss d'alignement :

$$L(B) = \sum_l |\mu_l(B) - \hat{\mu}_l| + |\sigma_l^2(B) - \hat{\sigma}_l^2|.$$

- 1 Choisir quelques couches (idéalement après normalisation, avant ReLU).
- 2 Collecter $\hat{\mu}_I, \hat{\sigma}_I^2$ sur le train (ou un set non shifté).
- 3 Au test : pour chaque batch, calculer les stats actuelles, appliquer la loss d'alignement, backprop sur tout le réseau.
- 4 Pas besoin de labels de test, ni de tâche SSL spéciale.

- Très bons résultats sur CIFAR-10C/100C, ImageNet-C.
- Fonctionne aussi pour la détection (YOLOv3 sur KITTI-Fog) et pour ViT.
- Méthode simple à implémenter, bien adaptée à un projet TER avec un modèle déjà pré-entraîné.

Comparaison entre TTT+ et ActMAD

Critère	TTT++	ActMAD
Besoin SSL spécial	Oui	Non
Alignement	Moments de features globaux	Moy/var par activation multi-couches
Domaines	surtout classification	cls + détection + régression
Complexité code	Élevée	Moyenne
Adapté TER	Théorie riche	Implémentation plus sûre

- Modèle de départ : backbone auto-supervisé + fine-tuning pour la classification.
- Méthode TTT principale envisagée : **ActMAD**, pour sa simplicité et sa généralité.
- Inspirations de TTT++ :
 - montrer que TTT naïf peut nuire aux performances;
 - l'idée de résumer la distribution source pour régulariser l'adaptation.

Plan expérimental du TER

- Baseline : modèle auto-supervisé + fine-tuning, sans TTT.
- Implémentation d'ActMAD au test-time.
- Comparaisons possibles :
 - pas de TTT,
 - simple adaptation BatchNorm,
 - ActMAD (et éventuellement une version simplifiée de TTT++).

- TTT++ met en avant le rôle de la self-supervision forte et du contrôle de la distribution des features.
- ActMAD propose un TTT simple et générique par alignement d'activations.
- Le TER cherche à évaluer, dans un cadre auto-supervisé, ce que ces idées apportent réellement en pratique.